THE OPEN SOURCE AUDIT

A Capstone Project, OSS NGMC Course

Chosen Software: Git

Student Name : Yashaswi Suman
Reg. Number : 24BEC10150
Slot : D11
Date of Submission : 31/03/2026

Part A, Origin and Philosophy (Units 1 & 2)

A1. The Problem Git Was Created to Solve

To understand why Git exists, you need to go back to 2002. The Linux kernel, one of the largest and most complex open-source projects in the world, had been growing rapidly, with hundreds of contributors submitting patches from around the globe. Managing those contributions was a logistical nightmare using nothing but emailed patches and Tarballs . Linus Torvalds, the creator of Linux, eventually adopted a proprietary distributed version control system called Bit Keeper, developed by Larry McVoy's company Bit Mover. Bit Keeper was offered to the Linux community for free, but it was not open-source software.

The situation unraveled in April 2005. A dispute arose over the reverse-engineering of Bit Keeper's protocol by Andrew Tridgell, a developer also known for creating Samba. Bit Mover responded by withdrawing free access to Bit Keeper for the Linux development community. Torvalds was left with a critical problem: the kernel's workflow depended entirely on a tool the community no longer had access to, and that tool was proprietary. He could not inspect it, modify it, or redistribute it. This was precisely the kind of situation the free software movement warned against: a dependence on someone else's locked tooling for work that needed to be free.

Torvalds made a characteristic decision. Rather than adopting another existing system, CVS was widely used but notoriously slow and poorly suited to distributed development, and Subversion was similarly centralised, he decided to write something better himself. In a matter of days in April 2005, he had the first version of Git working. 'I'm an egotistical bastard,' he later said, 'and I name all my projects after myself. First Linux, now Git.' The name 'git' is British slang for a foolish or worthless person, Torvalds chose it, apparently, to match his mood at the time.

The design goals Torvalds set for Git were directly shaped by what had frustrated him about existing tools. He wanted speed, patching the Linux kernel involved thousands of files, and CVS-style systems were painfully slow. He wanted true distributed development, every clone of a repository should be a complete, fully functional copy, not merely a checkout of a central server. He wanted strong data integrity, every file and commit in Git is check summed using SHA-1 hashes, making it impossible to silently corrupt data without Git knowing. And he wanted support for non-linear development, the complex branching and merging workflows that kernel development demanded.

What makes Git's origin story philosophically significant is not just that it was built quickly or cleverly. It is that it was built at all, and built openly, because proprietary control over a critical tool had directly harmed a community. Git did not emerge from a corporate product roadmap. It emerged from necessity, frustration, and the belief that infrastructure for collaborative work should itself be free and open. Within a few months, Torvalds handed maintenance over to Junio C. Hamano, who continues to steward the project today. Git's creation is, in miniature, the story of why open source matters.

A2. The License, What It Actually Says

The Four Freedoms of Free Software

Richard Stallman's GNU project defined four essential freedoms that a piece of software must grant in order to qualify as 'free software' in the philosophical sense. These are not four features or four nice-to-haves, they are the minimum conditions under which software can be considered to respect the user's autonomy:

- Freedom 0: The freedom to run the program for any purpose, without restriction.
- Freedom 1: The freedom to study how the program works and modify it to suit your needs. This requires access to the source code.
- Freedom 2: The freedom to redistribute copies of the program to help others.
- Freedom 3: The freedom to distribute copies of your modified versions to others, so the whole community can benefit.

Git is licensed under the GNU General Public License, version 2 (GPL v2). The GPL v2 grants all four of these freedoms fully. Anyone can download Git's source code from kernel.org or GitHub, read every line of it, modify it, redistribute it, and publish their own modified version, provided they do so under the same GPL v2 terms. There are no carve-outs, no enterprise tiers, and no features hidden behind a paywall.

Can a Company Modify Git and Sell It Without Sharing Changes?

No, and this is the defining characteristic of the GPL. The GPL is a 'copyleft' license, which means that any derivative work distributed to others must itself be distributed under the same GPL terms. If a company were to take Git's source code, modify it, and ship the modified version to customers, they would be legally required to provide those customers with the modified source code. They cannot 'close' the code.

This is what distinguishes the GPL from permissive licenses. Under a permissive license like the MIT license, a company can take the code, modify it, compile it, and ship a binary to customers with no obligation to share the changes. Under the GPL, the obligation to share 'propagates' with the code, hence the term 'viral' that critics sometimes use, and 'copyleft' that supporters prefer.

There is an important nuance, however. The GPL's redistribution requirements are triggered only when software is distributed to others. A company can run a modified version of Git on its own internal servers, using it for internal development, without any obligation to publish those changes. The GPL protects recipients of distributed software; it does not force companies to publish internal tools.

GPL vs MIT: Which Would You Choose?

The GNU General Public License (GPL) and the MIT License represent two philosophically distinct approaches to open-source software. The GPL is a copyleft license: it requires that derivative works also be distributed under the GPL, ensuring that freedom is perpetually preserved. The MIT license is permissive: it allows anyone to do almost anything with the software, including incorporating it into proprietary products, with no obligation to share modifications.

If I were writing a core development tool, something intended to be infrastructure for a community, I would choose the GPL, for the same reason Torvalds chose it for both the Linux kernel and Git. The GPL ensures that companies benefiting from the tool cannot privatise it. It maintains a level of

reciprocity: if you improve the tool, you improve it for everyone. However, if I were writing a library intended to be used widely across different projects, including commercial ones, I might choose the MIT or Apache license, because permissive licensing reduces friction and encourages adoption. The choice is not about which is 'better' in the abstract, but what relationship you want between your code and the commercial world.

Free as in Beer vs Free as in Freedom

The distinction between 'free as in free beer' and 'free as in freedom' is one of the foundational ideas in open-source philosophy, often attributed to Richard Stallman. 'Free as in free beer' means zero cost, you can obtain the software without paying. 'Free as in freedom' means you have the rights described by the four freedoms above: to run, study, modify, and redistribute.

These are not the same thing. A piece of software can be free to download but entirely proprietary, you cannot see its source code, cannot modify it, and cannot redistribute it. Conversely, software can cost money and still be free in the freedom sense, you could sell copies of Git legally, as long as you provided the source code and preserved the GPL terms.

Git itself illustrates this clearly. Git is free to download from git-scm.com at no cost, free as in free beer. But more importantly, it is free as in freedom: the complete source code is publicly available, any developer can study how the SHA-1 hashing works, any company can fork Git and build a custom version, and any user can redistribute it. These freedoms are what make Git genuinely open infrastructure rather than merely a free product.

A3. The Ethics of Open Source

Should All Software Be Open Source?

The argument for making all software open source is compelling in many domains. Software that runs critical public infrastructure, hospitals, election systems, government services, arguably should be open so that citizens, auditors, and security researchers can verify it is doing what it claims. The principle that 'code that affects everyone should be inspectable by everyone' has genuine democratic resonance. Open source also encourages collaborative improvement: a bug found by one person benefits all users.

However, the argument against mandatory open source is equally real. Businesses invest significant capital in developing software, and proprietary licensing is often what makes that investment viable. A startup writing a novel algorithm cannot be expected to give it away, the ability to monetise innovation is what funds innovation. Moreover, not all software is community infrastructure. A private company's internal tool for managing inventory has no obvious obligation to be public. The ethics of openness are contextual: they are strongest where software affects the public, and weaker where it is genuinely private.

The most defensible position is probably this: software that constitutes shared infrastructure, or that was built on public funding or community contribution, carries a strong ethical obligation toward openness. Software built privately with private capital for private use does not.

Is It Ethical to Profit from Free Community Labor?

This is one of the most contested questions in the open-source world. Companies like Red Hat, Google, and Meta have built enormously valuable businesses on top of Linux, which was built primarily by volunteer contributors and community developers. Is this ethical?

The case that it is ethical rests on the GPL's own terms. The license explicitly permits commercial use. When contributors publish their code under the GPL, they do so knowing that companies may use it. Red Hat's model, providing support, certification, and integration services around Linux, can be seen as a legitimate exchange: they contribute engineering resources back to the kernel while monetising the services around it.

The case that it raises ethical concerns focuses on asymmetry. Large technology companies employ relatively small numbers of contributors compared to the value they extract from open-source software. A volunteer who spent years improving a tool may find that the primary beneficiaries are corporations they have no relationship with. The 'standing on the shoulders of giants' metaphor cuts both ways: open source enables the giants, but the original builders sometimes never receive credit or compensation.

In the context of Git specifically, GitHub, which Microsoft acquired for 7.5 billion dollars, has built one of the world's most valuable developer platforms almost entirely around Git. Git itself is still maintained by a small, volunteer-led team. The ethical tension here is real and unresolved.

Does Open Source Enable or Undermine Original Innovation?

'Standing on the shoulders of giants' describes the cumulative nature of knowledge: each generation builds on what the previous one discovered. In software, this is unusually literal. Every program depends on libraries, compilers, operating systems, and protocols built by others. Open source makes those foundations accessible to everyone, not just those who can afford proprietary licenses.

Git is itself a perfect example of enabled innovation. It was built on existing ideas from Bit Keeper and other version control systems, written in C on top of Linux, and released freely. On top of Git, the world-built GitHub, GitLab, Bitbucket, CI/CD pipelines, automated deployment systems, and workflows that transformed how software is made. None of that would have been possible if Git had been proprietary.

The concern that open source undermines innovation is weaker than it first appears. Companies do not generally stop innovating because their foundational tools are open, if anything, having reliable, inspectable foundations allows them to focus creative effort higher up the stack. The real risk is not that open-source kills innovation, but that the contributors who build the foundations are not adequately supported, which is a sustainability problem, not an innovation problem.

Part B, Linux Footprint (Unit 2)

Installation Method

Git is available through the standard package managers on all major Linux distributions and does not need to be compiled from source for typical use. On Debian/Ubuntu-based systems, it is installed with:

```
sudo apt update && sudo apt install git
```

On RHEL/Fedora/CentOS-based systems:

```
sudo dnf install git
```

For users who need the latest version or want to examine Git's internals, it can also be compiled from source code downloaded from kernel.org. This requires GCC, make, and several libraries including libcurl and libssl. Compiling from source is instructive because it reveals Git's dependency chain and the flexibility that open-source code provides, a developer can, for example, compile Git with custom patches or without certain optional features.

Key Directories

Once installed, Git distributes itself across several standard Linux filesystem locations following the Filesystem Hierarchy Standard (FHS). The main executable lives at /usr/bin/git. Supporting executables, sub-commands like git-fetch, git-merge, and git-upload-pack, are stored in /usr/lib/git-core/ or /usr/libexec/git-core/ depending on the distribution. Documentation is installed into /usr/share/doc/git/ and manual pages into /usr/share/man/.

Unlike a server daemon, Git does not have a system-wide configuration file in /etc/ for its core operation. Its global user configuration lives in the user's home directory at ~/.gitconfig, and each repository stores its own configuration in .git/config within the repository root. The .git/ directory is the heart of any repository: it contains the object store (.git/objects/), references (.git/refs/), the HEAD pointer, hooks (.git/hooks/), and the index file that tracks the staging area.

User and Group, Security Implications

Git is not a daemon and does not run as a privileged system user for normal development use. It runs as whichever user invokes it from the shell. This is a deliberate and important security design: Git operations should have exactly the permissions of the user performing them, no more.

However, when Git is used as a server, for example, running git daemon or using SSH-based access to a shared repository, a dedicated system user is typically created. A common convention is to create a user named git with a home directory of /home/git or /srv/git. This user owns the bare repositories on the server and is the account through which SSH authentication is routed. Restricting the git user's shell to git-shell (a restricted shell that only allows Git operations) is a standard hardening measure.

The security rationale for running services as a dedicated low-privilege user is fundamental Linux hygiene: if the service is compromised, the attacker gains only the permissions of that user, not root,

and not access to other users' files. This principle, known as the principle of least privilege, is a recurring theme in how open-source software is designed to live on Linux systems.

Service Management

For standard developer use, Git is a command-line tool, not a persistent service, and does not require systemd management. However, git daemon, which provides unauthenticated read access to repositories over the git:// protocol, can be run as a systemd service. A sample unit file for git daemon would look as follows:

```
[Unit]
Description=Git Daemon
[Service]
ExecStart=/usr/lib/git-core/git-daemon --reuseaddr --base-path=/srv/git /srv/git
User=git
```

The usual systemctl commands apply:

```
sudo systemctl start git-daemon   sudo systemctl stop git-daemon   sudo systemctl enable git-daemon
```

Update Model

Security updates for Git reach end users through their distribution's package management system. When a vulnerability is discovered, and there have been significant ones, including remote code execution vulnerabilities in certain git clone operations, the distribution's security team backports the fix to the packaged version and pushes it through the standard update mechanism. On Debian/Ubuntu, this means:

```
sudo apt upgrade git
```

The open-source nature of Git is directly relevant to this process. Because the source code is public, security researchers worldwide can audit it continuously. Vulnerabilities are reported through public mailing lists, fixed by the maintainer team in a transparent process, and patches are publicly visible before and after they are applied. This stands in contrast to proprietary tools, where users must trust the vendor to discover, acknowledge, and fix vulnerabilities, often without any public visibility into the process.

Part C, The FOSS Ecosystem (Units 3 & 4)

Dependencies: What Git Is Built On

Git is written primarily in C, which means its direct runtime dependencies are minimal and stable. It relies on the standard C library (glibc on Linux), OpenSSL or GnuTLS for HTTPS transport security, libcurl for HTTP/HTTPS transfers, and zlib for compression of Git's object store. Each of these is itself a foundational piece of open-source infrastructure maintained by its own community.

For builds from source, Git requires GCC or Clang, GNU make, and Perl (used by several of Git's higher-level scripts and tools like git-svn and git-send-email). The Perl dependency is a reminder of Git's roots: in its earliest days, several Git tools were written as Perl scripts before being gradually rewritten in C for performance and portability.

What Git Has Enabled and Inspired

The impact of Git on the software ecosystem is difficult to overstate. It did not merely replace existing version control systems, it changed how software development is organised. The distributed model Git pioneered made workflows like fork-and-pull-request possible, which in turn made open-source contribution accessible to millions of developers who previously would have needed commit access to contribute.

GitHub, launched in 2008 and acquired by Microsoft in 2018, is the most consequential platform built on top of Git. It added social features, forking, pull requests, issue tracking, wikis, that transformed Git from a command-line tool into a collaboration platform used by over 100 million developers. GitLab followed with a fully open-source, self-hostable alternative. Gitea, another open-source fork of a similar platform, followed suit.

Git also directly enabled the modern CI/CD ecosystem. Tools like Jenkins, GitHub Actions, GitLab CI, and CircleCI all treat the Git repository as the source of truth for triggering automated builds, tests, and deployments. Infrastructure-as-code tools like Terraform and Ansible are typically stored and versioned in Git. The entire DevOps movement is, in a meaningful sense, built on Git's foundation.

Git in the LAMP Stack and the Broader Web

Git does not sit inside the LAMP stack (Linux, Apache, MySQL, PHP) as a runtime component, it is a development and deployment tool. But in practice, Git is present on virtually every server in the modern web stack as the mechanism by which code reaches production. A standard deployment workflow might involve a developer pushing to a Git remote, triggering a CI pipeline that runs tests, and if they pass, deploying to a server by checking out the latest commit.

Many web applications are deployed using git pull directly on the server, or using tools like Capistrano or Deployer that wrap Git operations. Configuration management tools like Ansible use Git to pull roles and playbooks. Package managers including npm, pip, and Composer can install packages directly from Git repositories. Git's relationship to the web is not as a component of it, but as the connective tissue between development and deployment.

Community and Governance

Git's development takes place primarily through a public mailing list, git@vger.kernel.org, which has been active since Git's earliest days. Unlike many modern projects, Git does not use a pull-request workflow on GitHub for its own core development, patches are submitted by email, reviewed publicly, and applied by the maintainer. This reflects both historical inertia and a deliberate philosophy: the email workflow is decentralised, does not depend on any single platform, and is entirely open.

Junio C. Hamano has served as Git's primary maintainer since Torvalds handed off the project in 2005. Git does not have a formal foundation or nonprofit structure, it is maintained as part of the broader kernel.org community, with contributions from individuals at companies including Google, Microsoft, and GitLab. The governance is meritocratic and informal: patches are accepted based on technical quality and community consensus, not corporate affiliation. The annual Git Merge conference brings together contributors from across the ecosystem.

Part D, Open Source vs Proprietary (Critical Analysis)

The closest proprietary alternative to Git for large-scale version control is Perforce Helix Core, used extensively in enterprise software, game development, and hardware design. The comparison below is honest, it does not assume Git is always the better choice.

Dimension	Git (Open Source)	Perforce Helix Core (Proprietary)
Cost	Free. No licensing fees, no per-seat cost.	Commercial licensing required. Costs scale with team size.
Security (Code Audit)	Source code is fully public. Any researcher can audit it.	Source code is closed. Users must trust vendor security claims.
Support & Reliability	Community support via mailing list and Stack Overflow. No SLA.	Vendor support with SLAs available. Preferred for regulated industries.
Freedom to Modify	Full freedom under GPL v2. Can fork, modify, redistribute.	No modification permitted. Locked to vendor's release cycle.
Community vs Control	Governed by meritocracy. No single company controls direction.	Roadmap controlled entirely by vendor. Users have limited influence.
Overall Verdict	Excellent for nearly all development workflows. Industry standard.	Better for very large binary asset management (games, hardware design).

Conclusion and Recommendation

For the overwhelming majority of software development contexts, web applications, open-source projects, startups, and large technology companies alike, Git is the clear recommendation. Its distributed model, performance, and zero licensing cost make it practically unmatched. The fact that its source code is fully auditable under the GPL means that security researchers, enterprises, and governments can verify its behaviour rather than trusting a vendor. GitHub, GitLab, and Bitbucket, all built on Git, provide the hosting, collaboration, and CI/CD integration that organisations need without any proprietary lock-in.

There is one domain where a proprietary alternative like Perforce genuinely outperforms Git: the management of very large binary files in workflows like AAA game development or hardware simulation, where repositories can reach terabytes in size and binary assets change frequently. Git was not designed for this use case, and while extensions like Git LFS partially address it, Perforce's architecture handles it more gracefully. A thoughtful recommendation would acknowledge this: Git for most software development; specialised tools for binary-heavy workflows. As for contributing to Git: yes. The project is well-governed, the mailing list is welcoming to good-faith contributors, and improving a tool used by over 100 million developers is one of the highest-leverage contributions a developer can make to the open-source commons.

Section 4, Shell Script Tasks

Each script below is fully commented, ready to run on any Linux system, and demonstrating the shell scripting concepts specified in the assignment. Each should be saved as a plain .sh file with execute permissions set using chmod +x.

Script 1, System Identity Report

This script demonstrates the use of shell variables, command substitution using(), basic output formatting with echo, and reading distribution information from the standard /etc/os-release file. It acts as a welcome screen that identifies the system and its open-source licensing context.

```
#!/bin/bash
# Script 1: System Identity Report
# Author: [Your Name] | Course: Open Source Software
# Description: Displays a welcome screen with key system identity info

# --- Student Variables ---
STUDENT_NAME="[Your Name]"          # Replace with your name
SOFTWARE_CHOICE="Git"               # Chosen open-source software

# --- Gather System Information using command substitution ---
KERNEL=$(uname -r)                  # Kernel version
USER_NAME=$(whoami)                 # Current logged-in user
HOME_DIR=$(echoHOME)               # Home directory of current user
UPTIME=$(uptime -p)                 # Human-readable uptime
DATETIME=$(date "+%A, %d %B %Y, %H:%M:%S")  # Current date and time
DISTRO=$(grep PRETTY_NAME /etc/os-release | cut -d= -f2 | tr -d '"')

# --- Display formatted output ---
echo "==========================================================="
echo "   OPEN SOURCE AUDIT, Welcome Screen"
echo "   Student :STUDENT_NAME | Software :SOFTWARE_CHOICE"
echo "==========================================================="
echo ""
echo "  Distribution :DISTRO"
echo "  Kernel       :KERNEL"
echo "  User         :USER_NAME"
echo "  Home Dir     :HOME_DIR"
echo "  Uptime       :UPTIME"
echo "  Date/Time    :DATETIME"
echo ""
# --- License message for the OS ---
echo "  License Note : The Linux kernel is licensed under GPL v2."
echo "                 This means you are free to run, study, modify,"
echo "                 and redistribute it, as long as you preserve"
echo "                 the same freedoms for others."
echo "==========================================================="
```

Key concepts used: STUDENT_NAME and SOFTWARE_CHOICE are simple shell variables. KERNEL, USER_NAME, UPTIME, and DISTRO all use command substitution, the() syntax executes the command inside and captures its output as a string. The DISTRO line uses a pipe to chain grep,

cut, and tr to extract a clean distribution name from /etc/os-release. Output is formatted using echo with consistent spacing.

Script 2, FOSS Package Inspector

This script demonstrates conditional logic (if-then-else), the case statement for multi-branch decisions, and working with package manager commands to extract software metadata. It detects whether the system uses RPM or APT and queries accordingly.

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: [Your Name] | Course: Open Source Software
# Description: Checks if a FOSS package is installed and describes its philosophy

PACKAGE='git'  # The package to inspect

# --- Detect package manager and check installation ---
if command -v rpm &>/dev/null; then
    # RPM-based system (Fedora, CentOS, RHEL)
    if rpm -qPACKAGE &>/dev/null; then
        echo "$PACKAGE is INSTALLED on this RPM-based system."
        rpm -qiPACKAGE | grep -E 'Version|License|Summary'
    else
        echo "$PACKAGE is NOT installed. Install with: sudo dnf installPACKAGE"
    fi
elif command -v dpkg &>/dev/null; then
    # Debian/Ubuntu-based system
    if dpkg -lPACKAGE &>/dev/null | grep -q '^ii'; then
        echo "$PACKAGE is INSTALLED on this Debian-based system."
        dpkg -lPACKAGE | grepPACKAGE
    else
        echo "$PACKAGE is NOT installed. Install with: sudo apt installPACKAGE"
    fi
else
    echo "Unknown package manager. Cannot verify installation."
fi

echo ''
# --- Case statement: print philosophy note for each FOSS package ---
casePACKAGE in
    git)
        echo "Git: Born from necessity when proprietary tools failed the Linux
community."
        echo "     Linus Torvalds built it in 2005 to keep kernel development free."
        ;;
    httpd|apache2)
        echo "Apache: The open web server that made the internet accessible to
everyone."
        ;;
    mysql|mariadb)
        echo "MySQL/MariaDB: A dual-license story, open source at its core, commercial
at its edge."
        ;;
    firefox)
        echo "Firefox: A nonprofit browser fighting to keep the web open and user-
controlled."
        ;;
```

```
      vlc)
          echo "VLC: Born in a student dorm in Paris, plays anything, owned by no
corporation."
          ;;
      python3|python)
          echo "Python: Shaped entirely by community consensus, the language that belongs
to everyone."
          ;;
      *)
          echo "$PACKAGE: An open-source tool, free to use, study, modify, and share."
          ;;
  esac
```

Key concepts used: The outer if-elif-else block detects whether the system uses RPM or APT by checking for the presence of the rpm command using command -v, which returns success if the command exists. The inner if blocks use rpm -q or dpkg -l to test package installation status, redirecting output to /dev/null with &>/dev/null to suppress it. The case statement provides multi-branch matching on the package name with wildcard fallback using *).

Script 3, Disk and Permission Auditor

This script demonstrates the for loop iterating over an array, the use of ls -ld and awk to extract filesystem metadata, and du to measure directory sizes. It also checks for the Git configuration directory.

```
  #!/bin/bash
  # Script 3: Disk and Permission Auditor
  # Author: [Your Name] | Course: Open Source Software
  # Description: Audits permissions and disk usage of key system directories

  # Array of directories to audit
  DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp" "/usr/lib/git-core")

  echo "============================================"
  echo "   Directory Audit Report"
  echo "   Generated:(date)"
  echo "============================================"
  echo ""

  # --- For loop: iterate over each directory ---
  for DIR in "${DIRS[@]}"; do
      if [ -d "$DIR" ]; then
          # awk extracts fields 1 (permissions), 3 (owner), 4 (group) from ls -ld
          PERMS=$(ls -ld "$DIR" | awk '{print1,3,4}')
          # du measures disk usage; 2>/dev/null suppresses permission errors
          SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
          echo "Directory :DIR"
          echo "  Permissions/Owner :PERMS"
          echo "  Size              :SIZE"
          echo "---"
      else
          echo "DirectoryDIR does NOT exist on this system."
          echo "---"
      fi
  done
```

```
# --- Check Git-specific config directory ---
echo ""
echo "=== Git Configuration Check ==="

GIT_CONFIG="$HOME/.gitconfig"
if [ -f "$GIT_CONFIG" ]; then
    echo "Global Git config found at:GIT_CONFIG"
    PERMS=$(ls -l "$GIT_CONFIG" | awk '{print1,3,4}')
    echo "Permissions:PERMS"
else
    echo "No global .gitconfig found. Run: git config --global user.name \"Name\""
fi

GIT_CORE_DIR='/usr/lib/git-core'
if [ -d "$GIT_CORE_DIR" ]; then
    PERMS=$(ls -ld "$GIT_CORE_DIR" | awk '{print1,3,4}')
    echo "Git core directory permissions:PERMS"
fi
```

Key concepts used: DIRS is a bash array defined with parentheses and space-separated values. The for loop uses{DIRS[@]} to expand all elements. The -d test inside the if checks whether each path is a directory. awk '{print1,3,4}' extracts specific whitespace-delimited fields from ls output, field 1 is permissions, 3 is owner, 4 is group. The du -sh command produces human-readable sizes (K, M, G), and cut -f1 extracts the first tab-separated field.

Script 4, Log File Analyzer

This script demonstrates reading a file line by line using a while read loop, using command-line arguments (1 and 2), counter variables with arithmetic expansion, and printing the last matching lines with grep and tail.

```
#!/bin/bash
# Script 4: Log File Analyzer
# Author: [Your Name] | Course: Open Source Software
# Usage: ./log_analyzer.sh /var/log/syslog error
# Description: Counts keyword occurrences in a log file line by line

# Command-line arguments
LOGFILE=$1                # First argument: path to log file
KEYWORD=${2:-'error'}     # Second argument: keyword to search (default: 'error')
COUNT=0
MAX_RETRIES=3
RETRY=0

# --- Validate that the log file argument was provided ---
if [ -z "$LOGFILE" ]; then
    echo "Usage:0 <logfile> [keyword]"
    exit 1
fi

# --- Retry loop if the file does not yet exist ---
while [ ! -f "$LOGFILE" ] && [RETRY -ltMAX_RETRIES ]; do
    echo "File not found:LOGFILE. Retrying in 2 seconds... ($((RETRY+1))/$MAX_RETRIES)"
```

```
    sleep 2
    RETRY=$((RETRY + 1))
done

if [ ! -f "$LOGFILE" ]; then
    echo "Error: FileLOGFILE not found afterMAX_RETRIES attempts."
    exit 1
fi

# --- Check if file is empty ---
if [ ! -s "$LOGFILE" ]; then
    echo "Warning: Log fileLOGFILE is empty."
    exit 0
fi

# --- Read log file line by line using while read loop ---
while IFS= read -r LINE; do
    # Case-insensitive keyword match using grep -iq
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))  # Arithmetic expansion to increment counter
    fi
done < "$LOGFILE"

echo ""
echo "=== Log Analysis Summary ==="
echo "File    :LOGFILE"
echo "Keyword :KEYWORD"
echo "Matches :COUNT lines"
echo ""

# --- Print last 5 matching lines ---
if [COUNT -gt 0 ]; then
    echo "Last 5 matching lines:"
    echo "----------------------"
    grep -i "$KEYWORD" "$LOGFILE" | tail -5
fi
```

Key concepts used:1 and2 are positional parameters, the first and second words after the script name when invoked.{2:-'error'} uses bash parameter expansion: if2 is unset or empty, the default value 'error' is used. The while [! -f ...] loop provides retry logic. The main while IFS= read -r LINE loop reads the file one line at a time, IFS= prevents trimming of leading whitespace, and -r prevents backslash interpretation. COUNT=$((COUNT + 1)) uses arithmetic expansion. The final grep | tail -5 combination filters matching lines and returns only the last five.

Script 5, The Open Source Manifesto Generator

This script demonstrates interactive user input with read, string building using variable concatenation, writing output to a file using > and >>, the date command, and includes a note demonstrating the alias concept.

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: [Your Name] | Course: Open Source Software
# Description: Generates a personalised open source philosophy statement
```

```
  # --- Alias demonstration (note: aliases in scripts require shopt -s expand_aliases) --
-
  # In interactive shells, you might define: alias today='date +%d\ %B\ %Y'
  # In this script we use the date command directly for portability.

  echo "==================================================="
  echo "   The Open Source Manifesto Generator"
  echo "==================================================="
  echo ""
  echo "Answer three questions to generate your manifesto."
  echo ""

  # --- Interactive input using read ---
  read -p '1. Name one open-source tool you use every day: ' TOOL
  read -p '2. In one word, what does freedom mean to you?  ' FREEDOM
  read -p '3. Name one thing you would build and share freely: ' BUILD

  # --- Capture current date ---
  DATE=$(date '+%d %B %Y')
  AUTHOR=$(whoami)
  OUTPUT="manifesto_${AUTHOR}.txt"

  # --- Build the manifesto paragraph using string variables ---
  LINE1="Open Source Manifesto, Generated onDATE byAUTHOR"
  LINE2=""
  LINE3="Every day, I rely onTOOL, a piece of software that someone built,"
  LINE4="shared, and trusted others to improve. That act of sharing is not weakness;"
  LINE5="it is the foundation of how knowledge grows. To me, freedom meansFREEDOM."
  LINE6="It means the right to see what a tool does, to fix it when it breaks,"
  LINE7="and to pass it on better than you found it. One day, I want to build"
  LINE8="$BUILD, and when I do, I will share it openly, because the tools"
  LINE9="I depend on today were built by people who believed in the same thing."

  # --- Write manifesto to file using > (create/overwrite) ---
  echo "$LINE1" > "$OUTPUT"
  echo "$LINE2" >> "$OUTPUT"
  echo "$LINE3" >> "$OUTPUT"
  echo "$LINE4" >> "$OUTPUT"
  echo "$LINE5" >> "$OUTPUT"
  echo "$LINE6" >> "$OUTPUT"
  echo "$LINE7" >> "$OUTPUT"
  echo "$LINE8" >> "$OUTPUT"
  echo "$LINE9" >> "$OUTPUT"

  echo ""
  echo "==================================================="
  echo "Manifesto saved to:OUTPUT"
  echo "==================================================="
  echo ""
  cat "$OUTPUT"
```

Key concepts used: read -p displays an inline prompt and captures user input into a named variable. DATE=(date '+%d %B %Y') uses command substitution with a format string, %d is the day, %B is the full month name, %Y is the four-digit year. String variables are concatenated simply by placing them inside double-quoted echo strings. The > operator writes to the output file, creating it or overwriting it if it exists. Subsequent >> operators append to the same file. The alias concept is demonstrated in

the comment, aliases are a shell feature for creating shorthand commands, but they require special handling in non-interactive scripts.